



Class Notes: Session 3 - Git Branching

Basic Branching and Merging

Branching in Git allows developers to create independent lines of development within a repository, enabling parallel work on features, bug fixes, or experiments without affecting the main codebase. Merging integrates changes from one branch into another, combining work from different development paths.

- **Key Concepts:**

- **Branch:** A lightweight, movable pointer to a commit, allowing isolated development.
- **Merging:** Combines changes from one branch into another, typically into the `main` branch.
- **Fast-Forward Merge:** When the target branch has no unique commits, Git moves the pointer forward.
- **Three-Way Merge:** Resolves conflicts when both branches have diverged, creating a merge commit.

- **Workflow:**

- i. Create a branch for a new feature or fix.
- ii. Make changes and commit them on the branch.
- iii. Merge the branch into the main branch when complete.

- **Example:**

A developer creates a branch, makes changes, and merges it:

```
git checkout -b feature-login
echo "Login functionality" > login.py
git add login.py
git commit -m "Added login functionality"
git checkout main
git merge feature-login
```

- **Use Case:**

A team develops a new feature on a separate branch to avoid disrupting the stable `main` branch, merging it only after testing.

- **Scenario:**

A developer creates a branch to add a search feature to a web app, works on it independently, and merges it into `main` after peer review.

- **Source:** *Version Control with Git* by Jon Loeliger and Matthew McCullough.

Creating a New Branch

Creating a branch in Git is a lightweight operation that creates a new pointer to a commit, allowing independent development.

- **Commands:**
 - Create a branch: `git branch <branch-name>` .
 - Switch to the branch: `git checkout <branch-name>` .
 - Create and switch in one step: `git checkout -b <branch-name>` .

- **Example:**

A developer creates a new branch for a bug fix:

```
git checkout -b bugfix-auth
echo "Fixed authentication issue" > auth.py
git add auth.py
git commit -m "Fixed auth bug"
```

- **Use Case:**

A developer creates a branch to experiment with a new UI design without affecting the production-ready codebase.
- **Scenario:**

A student creates a branch named `assignment-update` to work on a class project's new feature, ensuring the `main` branch remains stable.
- **Source:** *Git* by Ryan Hodson.

Delete, Rename, and Merge Two Branches

Git provides commands to manage branches by deleting, renaming, or merging them.

- **Deleting a Branch:**
 - Command: `git branch -d <branch-name>` (deletes only if merged).
 - Force delete (unmerged): `git branch -D <branch-name>` .

- **Renaming a Branch:**

- Command: `git branch -m <old-name> <new-name>` (rename current branch).
- Rename another branch: `git branch -m <old-name> <new-name> .`

- **Merging Two Branches:**

- Switch to the target branch: `git checkout <target-branch> .`
- Merge the source branch: `git merge <source-branch> .`

- **Example:**

A developer renames, merges, and deletes a branch:

```
git branch -m feature-login user-login
git checkout main
git merge user-login
git branch -d user-login
```

- **Use Case:**

After completing a feature, a developer merges the branch into `main` , deletes it to keep the repository clean, and renames a branch for clarity.

- **Scenario:**

A team renames a branch from `temp-fix` to `hotfix-payment` for better clarity, merges it into `main` , and deletes it after testing.

- **Source:** *The Ultimate Beginners Guide Git* by Adney Ainley.

Basic Merging to Master Branch

Merging to the `main` (or `master`) branch integrates changes from a feature or bugfix branch into the primary codebase. This is typically done after testing and review to ensure stability.

- **Steps:**

- i. Ensure the feature branch is complete and committed:
`git commit -m "Feature complete" .`
- ii. Switch to `main` : `git checkout main .`
- iii. Merge the feature branch: `git merge <feature-branch> .`
- iv. Resolve conflicts if any (edit conflicting files, stage, and commit).
- v. Optionally delete the feature branch: `git branch -d <feature-branch> .`

- **Example:**

Merging a feature branch into `main` :

```
git checkout main
git merge feature-search
git branch -d feature-search
```

- **Use Case:**

A team merges a tested `payment-gateway` branch into `main` to deploy a new payment feature to production.

- **Scenario:**

A developer completes a branch for adding a contact form, merges it into `main` after team approval, and deletes the branch to maintain a clean repository.

- **Source:** *Version Control with Git* by Jon Loeliger and Matthew McCullough.

Classwork

1. Create a Branch:

- Initialize a new Git repository in a folder named `web-app`.
- Create a branch named `feature-homepage`.
- Add a file `index.html` with basic HTML content.
- Commit the changes with the message "Added homepage structure."

2. Merge and Delete:

- Switch to the `main` branch.
- Merge the `feature-homepage` branch into `main`.
- Delete the `feature-homepage` branch.

3. Rename a Branch:

- Create a branch named `temp-fix`.
- Rename it to `bugfix-navigation`.
- Verify the rename using `git branch`.

Session Test

1. Multiple Choice:

- What does the `git checkout -b <branch-name>` command do?
 - A) Deletes a branch
 - B) Creates and switches to a new branch

- C) Merges a branch
- D) Renames a branch
- **Answer:** B

2. **Short Answer:**

- Explain the difference between a fast-forward merge and a three-way merge in Git.

3. **Practical:**

- Create a new branch named `feature-button`, add a file `styles.css` with some CSS code, commit it, merge it into `main`, and delete the branch.

4. **True/False:**

- The `git branch -D` command safely deletes a branch only if it has been merged.
(False)

5. **Scenario-Based:**

- Describe how a developer can create a branch to fix a bug, make changes, merge it into the `main` branch, and clean up the repository by deleting the branch.